Crea un Asistente de Voz con el Modelo GPT de OpenAI y IBM Watson



Tiempo estimado necesario: 1 hora

Introducción

¡Bienvenido a este proyecto guiado sobre la creación de un asistente de voz utilizando las bibliotecas de voz de OpenAI e IBM Watson para incrustar! El proyecto guiado te llevará a construir un asistente virtual que puede recibir entrada de voz, convertirla a texto utilizando tecnología de reconocimiento de voz, enviar el texto al modelo GPT de OpenAI, recibir una respuesta, convertirla a voz utilizando tecnología de texto a voz y, finalmente, reproducirla al usuario. El asistente de voz tendrá un front-end receptivo utilizando HTML, CSS y JavaScript, y un back-end confiable utilizando Flask.

Haz clic aquí para jugar con una demostración de la aplicación final que crearás.

Al final del curso, tendrás un profundo entendimiento de los asistentes de voz y las habilidades para crear tu propio asistente potenciado por IA que puede comunicarse a través de entrada y salida de voz. También tendrás una sólida base en desarrollo web utilizando Python, Flask, HTML, CSS y JavaScript, ¡y una impresionante aplicación de pila completa terminada!

Antes de comenzar, proporcionemos un poco de contexto sobre cada tema.

OpenAI

OpenAI es una organización de investigación que tiene como objetivo promover y desarrollar inteligencia artificial amigable de una manera que beneficie a la

The content of this lab is licensed under Apache 2.0

son modelos de procesamiento de lenguaje natural de vanguardia. Utilizarás GPT en tu asistente para permitirle entender y responder a una amplia gama de entradas de usuario.

Bibliotecas de voz de IBM Watson para incrustar

Las bibliotecas de voz de IBM Watson® para incrustar son un conjunto de bibliotecas de texto a voz y de voz a texto en contenedores diseñadas para ofrecer a nuestros socios de IBM mayor flexibilidad para incorporar lo mejor de la tecnología de IBM Research® en sus soluciones. Ahora disponibles como IA incrustable, los socios obtienen mayores capacidades para construir aplicaciones de transcripción de voz y síntesis de voz más rápidamente y desplegarlas en cualquier entorno híbrido de múltiples nubes. Estas tecnologías permiten que el asistente se comunique con los usuarios a través de entrada y salida de voz.

Asistentes de voz

Un asistente virtual es un programa diseñado para simular la conversación con usuarios humanos, especialmente a través de Internet utilizando la voz humana natural. Los asistentes pueden ser utilizados en una variedad de industrias, incluyendo servicio al cliente, comercio electrónico y educación.

Python (Flask)

Python es un lenguaje de programación popular que se utiliza ampliamente en el desarrollo web y la ciencia de datos. Flask es un marco web para Python que facilita la construcción de aplicaciones web. Utilizarás Python y Flask para construir el backend de tu asistente de voz. Python es un lenguaje poderoso que es fácil de aprender y tiene un gran ecosistema de bibliotecas y marcos que se pueden aprovechar en proyectos como el tuyo.

HTML - CSS - Javascript

HTML (Lenguaje de Marcado de Hipertexto) es un lenguaje de marcado utilizado para estructurar contenido en la web. CSS (Hojas de Estilo en Cascada) es un lenguaje de hojas de estilo utilizado para describir la apariencia y el formato de un documento escrito en HTML. Javascript es un lenguaje de programación que se utiliza comúnmente para agregar interactividad a las páginas web. Juntas, estas tecnologías nos permiten construir un frontend visualmente atractivo e interactivo para tu asistente. Los usuarios podrán interactuar con el asistente de voz a través

The content of this lab is licensed under Apache 2.0

Objetivos de aprendizaje

Al final de este proyecto, podrás:

- Explicar los conceptos básicos de los asistentes de voz y sus diversas aplicaciones
- Configurar un entorno de desarrollo para construir un asistente utilizando Python, Flask, HTML, CSS y Javascript
- Implementar la funcionalidad de voz a texto para permitir que el asistente entienda la entrada de voz de los usuarios
- Integrar el asistente con el modelo GPT de OpenAI para darle un alto nivel de inteligencia y la capacidad de entender y responder a las solicitudes de los usuarios
- Implementar la funcionalidad de texto a voz para permitir que el asistente se comunique con los usuarios a través de salida de voz
- Combinar todos los componentes anteriores para crear un asistente funcional que pueda recibir entrada de voz y proporcionar una respuesta hablada
- (Opcional) Desplegar el asistente en un servidor web para su uso por un público más amplio

Requisitos previos

Tener conocimientos de los conceptos básicos de HTML/CSS, Javascript y Python es deseable, pero no esencial. Haremos nuestro mejor esfuerzo para explicar cada paso del proceso, así como cualquier código mostrado a lo largo del camino.

The content of this lab is licensed under Apache 2.0

Paso 1: Entendiendo la interfaz

En este proyecto, el objetivo es crear una interfaz que permita la comunicación con un asistente de voz, y un backend para gestionar el envío y recepción de respuestas.

El frontend utilizará HTML, CSS y Javascript con bibliotecas populares como Bootstrap para el estilo básico, Font Awesome para los íconos y JQuery para el manejo eficiente de acciones. La interfaz de usuario será similar a otras aplicaciones de asistentes de voz, como Google Assistant. El código para la interfaz se proporciona y el enfoque del curso está en construir el asistente de voz e integrarlo con varios servicios y APIs. El código proporcionado te ayudará a entender cómo interactúan el frontend y el backend, y a medida que lo revises, aprenderás sobre las partes importantes y cómo funciona, dándote una buena comprensión de cómo funciona el frontend y cómo crear esta página web simple.

Ejecuta los siguientes comandos para recibir el esquema del proyecto, renombrarlo para guardarlo con otro nombre y finalmente moverte a ese directorio.

```bash
git clone https://github.com/ibm-developer-skills-network/bkrva-chatapp-wi
mv bkrva-chatapp-with-voice-and-openai-outline chatapp-with-voice-and-open
cd chatapp-with-voice-and-openai-outline
```

Run

La siguiente sección ofrece una breve comprensión de cómo funciona el frontend.

HTML, CSS y Javascript

El archivo `index.html` es responsable del diseño y la estructura de la interfaz web. Este archivo contiene el código para incorporar bibliotecas externas como JQuery, Bootstrap y FontAwesome Icons, así como el CSS (`style.css`) y el código Javascript (`script.js`) que controlan el estilo y la interactividad de la interfaz.

El archivo `style.css` es responsable de personalizar la apariencia visual de los componentes de la página. También maneja la animación de carga utilizando

The content of this lab is licensed under Apache 2.0

animación en varios puntos en el tiempo, permitiendo una transición suave entre diferentes estilos y creando animaciones dinámicas.

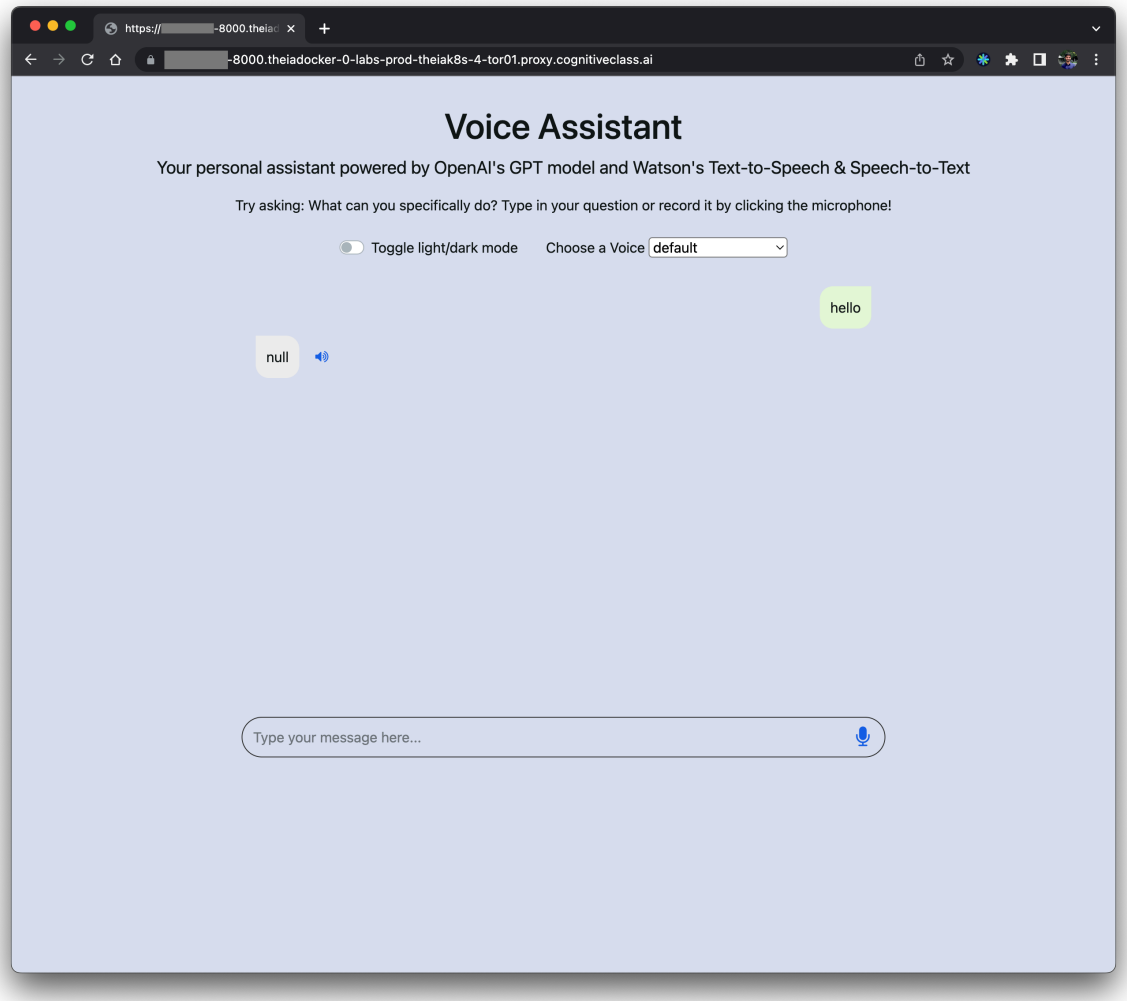
El archivo `script.js` es responsable de la interactividad y funcionalidad de la página. Contiene la mayor parte del código y maneja todas las funciones necesarias, como cambiar entre el modo claro y oscuro, enviar mensajes y mostrar nuevos mensajes en la pantalla. Incluso permite a los usuarios grabar audio.

Imágenes de la UI

Aquí hay algunas imágenes del frontend que recibiste.

Modo claro

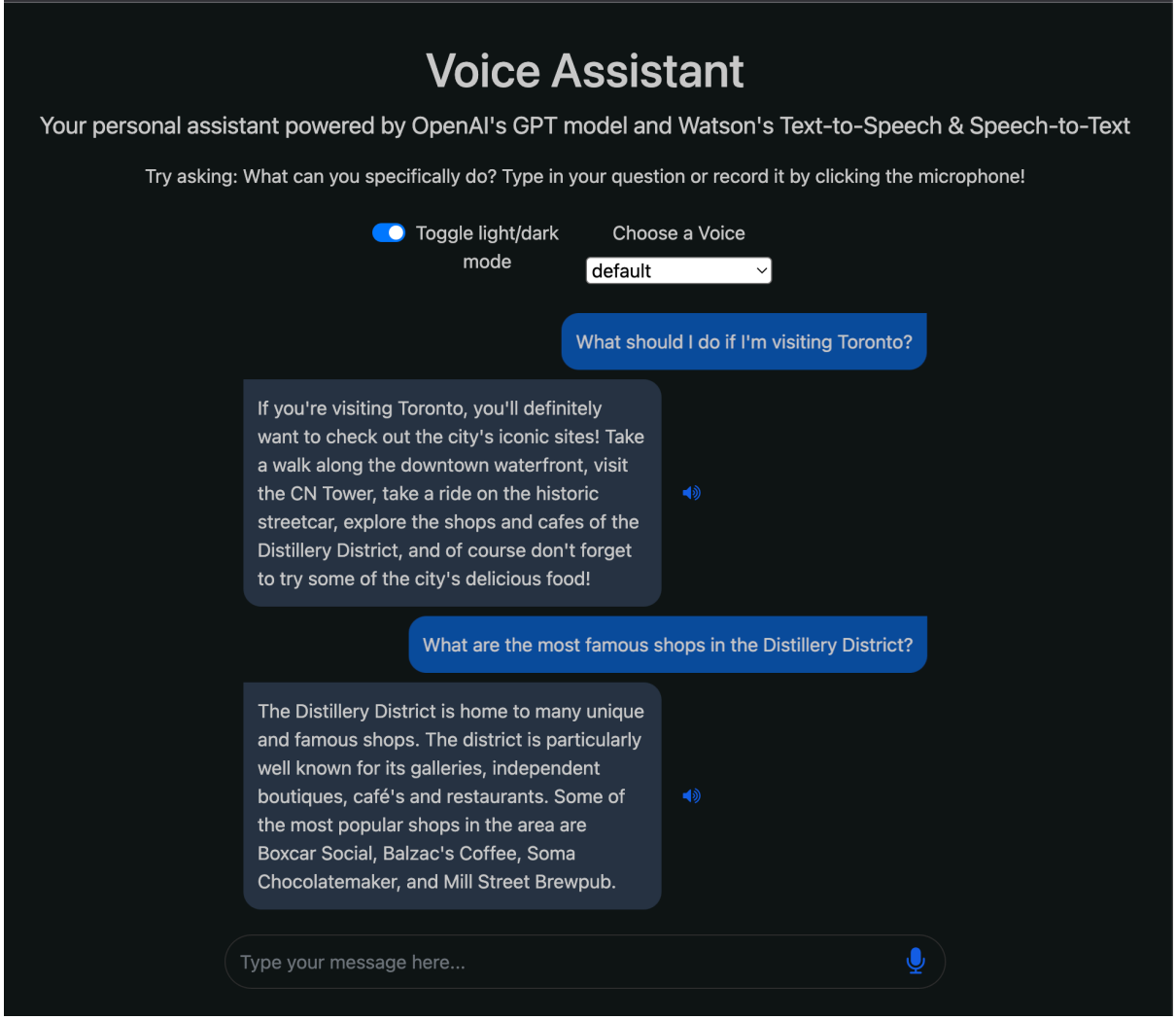
Esto demuestra cómo funciona el código base. Solo devolverá `null` como respuesta.



Modo oscuro

Una vez que completes el proyecto, el asistente podrá dar respuestas claras como

The content of this lab is licensed under Apache 2.0



The content of this lab is licensed under Apache 2.0

Paso 2: Entendiendo el servidor

El servidor es cómo la aplicación se ejecutará y se comunicará con todos tus servicios. Flask es un marco de desarrollo web para Python y puede ser utilizado como un backend para la aplicación. Es un marco ligero y simple que facilita la creación rápida y sencilla de aplicaciones web.

Con Flask, puedes crear páginas web y aplicaciones sin necesidad de conocer mucho código complejo o usar herramientas o bibliotecas adicionales. Puedes crear tus propias rutas y manejar las solicitudes de los usuarios, y también te permite conectarte a APIs y servicios externos para recuperar o enviar datos.

Este proyecto guiado utiliza Flask para manejar el backend de tu asistente de voz. Esto significa que estarás usando Flask para crear rutas y manejar solicitudes y respuestas HTTP. Cuando un usuario interactúa con el asistente de voz a través de la interfaz del frontend, la solicitud se enviará al backend de Flask. Flask procesará la solicitud y la enviará al servicio correspondiente.

El código proporcionado da el esquema para el servidor en el archivo `server.py`.

Open server.py in IDE

En la parte superior del archivo, hay varias declaraciones de importación. Estas declaraciones se utilizan para incluir bibliotecas y módulos externos, que se usarán en el archivo actual. Por ejemplo, `speech_to_text` es una función dentro del archivo `worker.py`, mientras que `openai` es un paquete que necesita ser instalado para usar el modelo GPT-3 de OpenAI. Estos paquetes, módulos y bibliotecas importados te permitirán acceder a las funcionalidades y métodos adicionales que ofrecen, facilitando la interacción con el modelo de reconocimiento de voz y GPT-3 en tu código.

Debajo de las importaciones, se inicializa la aplicación Flask y se establece una política de CORS. Una política de CORS se utiliza para permitir o prevenir que las páginas web realicen solicitudes a diferentes dominios que el que sirvió la página web. Actualmente, está configurada como `*` para permitir cualquier solicitud.

El archivo `server.py` consta de 3 funciones que se definen como rutas, y el código

The content of this lab is licensed under Apache 2.0

La primera ruta es:

```python
@app.route('/', methods=['GET'])
def index():
    return render_template('index.html')
```

Cuando un usuario intenta cargar la aplicación, inicialmente envía una solicitud para ir al endpoint `/`. Luego, activarán esta función `index` arriba y ejecutarán el código anterior. Actualmente, el código devuelto por la función es una función de renderizado para mostrar el archivo `index.html`, que es la interfaz del frontend.

Las segunda y tercera rutas son las que se utilizarán para procesar todas las solicitudes y manejar el envío de información entre las aplicaciones.

Finalmente, la aplicación se inicia con el comando `app.run` para ejecutarse en el puerto `8080` y tener el host en `0.0.0.0`.

Las siguientes secciones te guiarán a través del proceso de completar las funciones `process_prompt_route` y `speech_to_text_route` en este archivo y te ayudarán a entender cómo usar los paquetes y endpoints.

The content of this lab is licensed under Apache 2.0

Paso 3: Ejecutar la aplicación

Docker permite la creación de "contenedores" que empaquetan una aplicación y sus dependencias juntas. Esto permite que la aplicación se ejecute de manera consistente en diferentes entornos, ya que el contenedor incluye todo lo que necesita para funcionar. Además, usar una imagen de Docker para crear y ejecutar aplicaciones puede simplificar el proceso de implementación, ya que la imagen puede ser fácilmente distribuida y ejecutada en cualquier máquina que tenga Docker instalado. Esto puede ayudar a asegurar que la aplicación funcione de la misma manera en los entornos de desarrollo, pruebas y producción.

El `git clone` del Paso 1 ya viene con un `Dockerfile` y `requirements.txt` para esta aplicación. Estos archivos se utilizan para construir la imagen con las dependencias ya instaladas. Al mirar el `Dockerfile`, puedes ver que es bastante simple, solo crea un entorno de python, mueve todos los archivos del directorio local al contenedor, instala los paquetes requeridos y luego inicia la aplicación ejecutando el comando `python`.

Se necesitan ejecutar simultáneamente 3 contenedores diferentes para que la aplicación funcione e interactúe con las capacidades de Texto a Voz y Voz a Texto.

Pequeños requisitos previos:

Necesitas ejecutar estos comandos con un solo clic para cumplir con algunos de los requisitos previos:

bash

```
mkdir /home/project/chatapp-with-voice-and-openai-outline/certs/
cp /usr/local/share/ca-certificates/rootCA.crt /home/project/chatapp-with-
```

Run

1. Iniciar la aplicación

Esta imagen se construye rápidamente ya que la aplicación es bastante pequeña. Estos comandos primero construyen la aplicación (ejecutando los comandos en el `Dockerfile`) y etiquetan (nombran) el contenedor construido como `voice-`

The content of this lab is licensed under Apache 2.0

Necesitarás ejecutar estos comandos cada vez que desees hacer un nuevo cambio en uno de los archivos.

bash

```
docker build . -t voice-chatapp-powered-by-openai
docker run -p 8000:8000 voice-chatapp-powered-by-openai
```

Run

Abrir aplicación

La aplicación debe abrirse en una nueva pestaña ya que el minibrowser en este entorno no puede soportar ciertas características requeridas.

Es posible que tu navegador deniegue "ventanas emergentes", pero por favor, permítelas para que se abra la nueva pestaña.

En este punto, la aplicación se ejecutará pero devolverá `null` para cualquier entrada.

Una vez que hayas tenido la oportunidad de ejecutar y explorar la aplicación, presiona `Command + C` para Mac y `CTRL+C` para Windows al mismo tiempo para detener el contenedor y continuar con el proyecto.

La aplicación solo se ejecutará mientras el contenedor esté activo. Si realizas nuevos cambios en los archivos y deseas probarlos, tendrás que reconstruir la imagen.

2. Iniciando el reconocimiento de voz a texto

Skills Network proporciona su propia imagen de Watson Speech-to-Text que se ejecuta automáticamente en este entorno. Para acceder a ella, utiliza esta URL de endpoint cuando llegues al Paso 4:

python

```
base_url = "https://sn-watson-stt.labs.skills.network"
```

Puedes probar que funciona ejecutando esta consulta:

The content of this lab is licensed under Apache 2.0

```bash
curl https://sn-watson-stt.labs.skills.network/speech-to-text/api/v1/model
```

Run

Deberías ver una lista de algunos idiomas que puede reconocer. A continuación se muestra un ejemplo de salida.

```JSON
{
   "models": [
      {
         "name": "en-US_Multimedia",
         "language": "en-US",
         "description": "US English multimedia model for broadband audio (
          ...
      },
      {
         "name": "fr-FR_Multimedia",
         "language": "fr-FR",
         "description": "French multimedia model for broadband audio (16kH
          ...
      }
   ]
}
```

3. Iniciando Text-to-Speech

Skills Network proporciona su propia imagen de Watson Text-to-Speech que se ejecuta automáticamente en este entorno. Para acceder a ella, utiliza esta URL de endpoint cuando llegues al Paso 6:

```python
base_url = "https://sn-watson-tts.labs.skills.network"
```

Puedes probar que funciona ejecutando esta consulta:

The content of this lab is licensed under Apache 2.0

bash

```
curl https://sn-watson-tts.labs.skills.network/text-to-speech/api/v1/voice
```

Run

Deberías ver una lista de varias voces diferentes que este modelo puede usar. A continuación se muestra un ejemplo de salida.

JSON


```
{
   "voices": [
      {
         "name": "en-US_EmilyV3Voice",
         "language": "en-US",
         "gender": "female",
         "description": "Emily: American English female voice. Dnn technol
         ...
      },
      {
         "name": "en-GB_JamesV3Voice",
         "language": "en-GB",
         "gender": "male",
         "description": "James: British English male voice. Dnn technology
         ...
      },
      {
         "name": "en-US_MichaelV3Voice",
         "language": "en-US",
         "gender": "male",
         "description": "Michael: American English male voice. Dnn technol
         ...
      },
      {
         "name": "fr-CA_LouiseV3Voice",
         "language": "fr-CA",
         "gender": "female",
         "description": "Louise: French Canadian female voice. Dnn technol
         ....
      },
      ...
   ]
}
```


The content of this lab is licensed under Apache 2.0

Paso 4: Integración de Watson Speech-to-Text

La funcionalidad de Speech-to-Text es una tecnología que convierte el habla en texto utilizando aprendizaje automático. Es útil para la accesibilidad, la productividad, la conveniencia, el soporte multilingüe y soluciones rentables para una amplia gama de aplicaciones. Por ejemplo, poder tomar la voz de un usuario como entrada para una aplicación de chat.

Usando el modelo de IA de Watson Speech-to-Text integrado que se desplegó anteriormente, es posible convertir fácilmente tu voz en texto mediante una simple API. Este resultado se puede pasar a la API de OpenAI para generar una respuesta.

Implementación

Actualizarás una función llamada `speech_to_text` que tomará los datos de audio recibidos del navegador y los pasará a la API de Watson Speech-to-Text. Abre `worker.py` desde el explorador o haciendo clic abajo.

Open worker.py in IDE

Es importante importar la biblioteca requests en la parte superior de tu archivo `worker.py`. Esta biblioteca es una simple biblioteca de solicitudes HTTP que utilizarás para hacer llamadas a la API de Watson Speech-to-Text.

La función `speech_to_text` tomará los datos de audio como parámetro, hará una llamada a la API de Watson Speech-to-Text utilizando la biblioteca requests y devolverá la transcripción de los datos de audio.

Recuerda reemplazar el `...` de la variable `base_url` con la URL de tu modelo de Speech-to-Text (por ejemplo, `https://sn-watson-stt.labs.skills.network`).

The content of this lab is licensed under Apache 2.0

Worker.py

python

```
def speech_to_text(audio_binary):

    # Set up Watson Speech-to-Text HTTP Api url
    base_url = '...'
    api_url = base_url+'/speech-to-text/api/v1/recognize'

    # Set up parameters for our HTTP request
    params = {
        'model': 'en-US_Multimedia',
    }

    # Set up the body of our HTTP request
    body = audio_binary

    # Send a HTTP Post request
    response = requests.post(api_url, params=params, data=audio_binary).js

    # Parse the response to get our transcribed text
    text = 'null'
    while bool(response.get('results')):
        print('speech to text response:', response)
        text = response.get('results').pop().get('alternatives').pop().get
        print('recognised text: ', text)
        return text
```

Explicación

La función simplemente toma `audio_binary` como el único parámetro y luego lo envía en el cuerpo de la solicitud HTTP.

Para hacer una solicitud HTTP Post a la API de Watson Speech-to-Text, necesitas lo siguiente:

1. **URL** de la API: Esto se define como `api_url` en tu código y apunta al servicio de Speech-to-Text de Watson.
2. **Parámetros**: Esto se define como `params` en tu código. Es simplemente un diccionario que tiene un par clave-valor, es decir, `'model': 'en-US_Multimedia'` que le dice a Watson que deseas utilizar el modelo de inglés estadounidense para procesar tu discurso.

The content of this lab is licensed under Apache 2.0

3. **Cuerpo** de la solicitud: esto se define como `body` y es igual a `audio_binary`, ya que estás enviando los datos de audio dentro del cuerpo de tu solicitud POST.

Luego usas la biblioteca requests para enviar esta solicitud HTTP pasando la url, params y data(cuerpo) a ella y luego usas `.json()` para convertir la respuesta de la API a formato json, que es muy fácil de analizar y puede ser tratado como un diccionario en Python.

La estructura de la respuesta es como se muestra a continuación:



json



```
{
  "results": [
    {
      "alternatives": [
        {
          "transcript": "Recognized text from your speech",
          "confidence": 0.98
        }
      ]
    }
  ]
}
```

Por lo tanto, verificas si la respuesta contiene algún resultado, y si es así, extraes el texto obteniendo la cadena de transcripción anidada como se mostró arriba. Luego, devuelves este texto.

Consejo pequeño

Observa las declaraciones de impresión como *print('response', response)*, siempre es una buena idea imprimir los datos que estás recibiendo de algún lugar externo como una API en este caso, ya que realmente ayuda con la depuración si algo sale mal.

The content of this lab is licensed under Apache 2.0

Paso 5: Integrando la API de OpenAI

¡Es hora de darle un cerebro a tu asistente de voz! Con el poder de la API GPT de OpenAI, puedes enviar el texto transcrito y recibir respuestas que respondan a tus preguntas.

Normalmente, necesitarías obtener una clave API creando una cuenta en OpenAI, sin embargo, cuando uses nuestros laboratorios, esto ya ha sido gestionado para que puedas continuar con tu proyecto sin preocuparte por ello.

Función de procesamiento de mensajes de OpenAI

Actualizarás la función llamada `openai_process_message`, que tomará un mensaje y lo pasará a la API GPT de OpenAI para recibir una respuesta. Esencialmente, es el equivalente a presionar el botón de enviar para obtener una respuesta de ChatGPT.

Adelante, actualiza la función `openai_process_message` en el archivo `worker.py` con lo siguiente:

Open worker.py in IDE

The content of this lab is licensed under Apache 2.0

Worker.py

python

```python
def openai_process_message(user_message):
    # Establecer el prompt para la API de OpenAI
    prompt = "Actúa como un asistente personal. Puedes responder preguntas
    # Llamar a la API de OpenAI para procesar nuestro prompt
    openai_response = openai_client.chat.completions.create(
        model="gpt-5-nano",
        messages=[
            {"role": "system", "content": prompt},
            {"role": "user", "content": user_message}
        ],
        max_completion_tokens=1000
    )
    print("respuesta de openai:", openai_response)
    # Analizar la respuesta para obtener el mensaje de respuesta para nues
    response_text = openai_response.choices[0].message.content
    return response_text
```

Explicación

La función es realmente simple, gracias a la biblioteca `openai`, que es muy fácil de usar.

Aquí es donde puedes darle algo de personalidad a tu asistente personal. En este caso, le estás diciendo al modelo que actúe como un asistente personal al: `Actúa como un asistente personal`, y luego le das tareas específicas que puede realizar: `Puedes responder preguntas, traducir oraciones, resumir noticias y dar recomendaciones.`. Al añadir el mensaje original del usuario después, le das a OpenAI más espacio para sonar genuino. Siéntete libre de cambiar esto según lo que necesites.

Luego llamas a la API de OpenAI utilizando la función `openai.chat.completions.create` y pasas los siguientes 3 parámetros:

1. `model`: Este es el modelo de OpenAI que queremos usar para procesar nuestro prompt.
2. `messages`: El parámetro messages es un array de objetos utilizados para definir el flujo de conversación entre el usuario y la IA. Cada objeto representa un mensaje con dos atributos clave: role (identificando al remitente como

The content of this lab is licensed under Apache 2.0

usuario) y content (el texto del mensaje). El mensaje de rol "system" instruye a la IA sobre cómo comportarse (por ejemplo, actuando como un asistente personal), mientras que el mensaje de rol "user" contiene la entrada del usuario. Este enfoque estructurado ayuda a personalizar las respuestas de la IA para que sean más relevantes y personalizadas.

3. `max_completion_tokens`: Esto define el número máximo de tokens que el modelo puede producir en su respuesta. Con una configuración de 1000, la respuesta puede ser bastante detallada.

Nuevamente, definitivamente puedes ajustar estos parámetros según tus necesidades personalizadas y puedes aprender más sobre ellos visitando OpenAI playground, donde puedes probar todos los parámetros en tiempo real.

La estructura de la respuesta es algo así:

```json
{
  "choices": [
    {
      "message": {
        "content": "The model's answer to our prompt"
      }
    }
  ]
}
```

Por lo tanto, analizas la `openai_response` para extraer la respuesta a tu solicitud, que es `openai_response.choices[0].message.content` y la almacenas en una variable llamada `response_text`. Finalmente, devuelves el `response_text`.

The content of this lab is licensed under Apache 2.0

Paso 6: Integración de Watson Text-to-Speech

Es hora de darle una voz a tu asistente utilizando la funcionalidad de Text-to-Speech.

Una vez que hayamos procesado el mensaje del usuario utilizando OpenAI, agreguemos la función final del trabajador que convertirá esa respuesta en voz, para que tengas una sensación más personalizada ya que el Asistente Personal leerá la respuesta para ti. Al igual que otros asistentes virtuales como Google, Alexa, Siri, etc.

Función Text-to-Speech

En el archivo `worker.py`, la función `text_to_speech` envía datos a la API de Text-to-Speech de Watson para obtener los datos como salida hablada.

Esta función será muy similar a `speech_to_text`, ya que utilizarás nuevamente tu biblioteca de solicitudes para hacer una solicitud HTTP. Vamos a sumergirnos en el código.

Recuerda reemplazar el `...` en la variable `base_url` con la URL de tu modelo de Text-to-Speech (por ejemplo, `https://sn-watson-tts.labs.skills.network`).

The content of this lab is licensed under Apache 2.0

Worker.py

```python
def text_to_speech(text, voice=""):
    # Configurar la URL de la API HTTP de Watson Text-to-Speech
    base_url = '...'
    api_url = base_url + '/text-to-speech/api/v1/synthesize?output=output_

    # Agregar el parámetro de voz en api_url si el usuario ha seleccionado
    if voice != "" and voice != "default":
        api_url += "&voice=" + voice

    # Establecer los encabezados para nuestra solicitud HTTP
    headers = {
        'Accept': 'audio/wav',
        'Content-Type': 'application/json',
    }

    # Establecer el cuerpo de nuestra solicitud HTTP
    json_data = {
        'text': text,
    }

    # Enviar una solicitud HTTP Post al Servicio Watson Text-to-Speech
    response = requests.post(api_url, headers=headers, json=json_data)
    print('respuesta de texto a voz:', response)
    return response.content
```

Explicación

La función simplemente toma `text` y `voice` como parámetros. Agrega la voz como un parámetro al `api_url` si no está vacío o no es predeterminado. Envía el `text` en el cuerpo de la solicitud HTTP.

De manera similar a antes, para hacer una solicitud HTTP Post a la API de Watson Text-to-Speech, necesitas los siguientes tres elementos:

1. **URL** de la API: Esto se define como `api_url` en tu código y apunta al servicio de Text to Speech de Watson. Esta vez también agregas un parámetro de voz al `api_url` si el usuario ha enviado una voz preferida en su solicitud.
2. **Encabezados**: Esto se define como `headers` en tu código. Es solo un diccionario que tiene dos pares clave-valor. El primero es *'Accept':'audio/wav'* que especifica que esperamos que el servidor devuelva audio de salida en

The content of this lab is licensed under Apache 2.0

formato WAV. El segundo es 'Content-Type':'application/json', lo que significa que el formato del cuerpo será *JSON*.

3. **Cuerpo** de la solicitud: Esto se define como `json_data` y es un diccionario que contiene el par clave-valor 'text': `text`, este texto será procesado y convertido en voz.

Luego utilizamos la biblioteca requests para enviar esta solicitud HTTP, pasando la URL, los encabezados y el JSON (cuerpo). La respuesta devuelta por la API es datos de audio binario, por lo que retornamos response.content.

Por lo tanto, devolvemos `response.content` que contiene los datos de audio recibidos.

***NOTA:* Puedes encontrar errores de indentación en archivos de Python. Asegúrate de que tu código utilice un espaciado consistente (sin mezclar tabulaciones y espacios) y verifica la indentación cuidadosamente antes de ejecutar la aplicación.**

The content of this lab is licensed under Apache 2.0

Paso 7: Uniendo todo creando puntos finales de API Flask

Ahora, utilizando las funciones que definimos en las secciones anteriores, podemos conectar todo y completar el asistente.

Los cambios en esta sección serán para el archivo server.py.

Open server.py in IDE

El esquema ya se ha encargado de las importaciones de las funciones del archivo `worker.py` al archivo `server.py`. Esto permite que el archivo `server.py` acceda a estas funciones importadas del archivo `worker.py`.

python

```python
from worker import speech_to_text, text_to_speech, openai_process_message
```

Ahora actualizaremos dos rutas de Flask, una para convertir el habla del usuario a texto (`speech_to_text_route`) y la otra para procesar su mensaje y convertir la respuesta de OpenAI de nuevo a habla (`process_message_route`).

Ruta de habla a texto

Esta función es muy simple, ya que solo convierte el habla del usuario a texto utilizando el `speech_to_text` que definimos en una de nuestras secciones anteriores y devuelve la respuesta.

The content of this lab is licensed under Apache 2.0

Código

```python
@app.route('/speech-to-text', methods=['POST'])
def speech_to_text_route():
    print("processing speech-to-text")
    audio_binary = request.data # Get the user's speech from their request
    text = speech_to_text(audio_binary) # Call speech_to_text function to

    # Return the response back to the user in JSON format
    response = app.response_class(
        response=json.dumps({'text': text}),
        status=200,
        mimetype='application/json'
    )
    print(response)
    print(response.data)
    return response
```

Explicación

Comenzamos almacenando `request.data` en una variable llamada `audio_binary`, ya que estamos enviando los datos binarios de audio en el cuerpo de la solicitud desde el front end. Luego usamos nuestra función previamente definida `speech_to_text` y pasamos `audio_binary` como parámetro. Almacenamos el valor de retorno en una nueva variable llamada `text`.

Dado que nuestro front end espera una respuesta JSON, creamos una respuesta json utilizando la función `app.response_class` de Flask y pasando tres argumentos:

1. `response`: Estos son los datos reales que queremos enviar en el cuerpo de nuestra respuesta HTTP. Usaremos la función `json.dumps` y pasaremos un diccionario simple que contiene solo un par clave-valor - `'text': text`
2. `status`: Este es el código de estado de la respuesta HTTP; lo estableceremos en 200, lo que significa que la respuesta es *OK* y que la solicitud ha tenido éxito.
3. `mimetype`: Este es el formato de nuestra respuesta que se escribe más formalmente como `'application/json'` en la solicitud/respuesta HTTP.

Luego regresamos la `response`.

The content of this lab is licensed under Apache 2.0

Ruta de procesamiento de mensajes

Esta función aceptará básicamente el mensaje de un usuario en forma de texto con su voz preferida. Luego utilizará nuestras funciones auxiliares previamente definidas para llamar a la API de OpenAI para procesar este aviso y luego convertir esa respuesta en habla utilizando la API de Texto a Voz de Watson, y devolver el resultado al usuario.

Código

python

```python
@app.route('/process-message', methods=['POST'])
def process_message_route():
    user_message = request.json['userMessage'] # Get user's message from t
    print('user_message', user_message)

    voice = request.json['voice'] # Get user's preferred voice from their
    print('voice', voice)

    # Call openai_process_message function to process the user's message a
    openai_response_text = openai_process_message(user_message)

    # Clean the response to remove any emptylines
    openai_response_text = os.linesep.join([s for s in openai_response_tex

    # Call our text_to_speech function to convert OpenAI Api's reponse to
    openai_response_speech = text_to_speech(openai_response_text, voice)

    # convert openai_response_speech to base64 string so it can be sent ba
    openai_response_speech = base64.b64encode(openai_response_speech).deco

    # Send a JSON response back to the user containing their message's res
    response = app.response_class(
        response=json.dumps({"openaiResponseText": openai_response_text, "
        status=200,
        mimetype='application/json'
    )

    print(response)
    return response
```

Explicación

Comenzaremos almacenando el mensaje del usuario en `user_message` utilizando

The content of this lab is licensed under Apache 2.0

preferida del usuario en `voice` utilizando `request.json['voice']`.

Luego utilizaremos la función auxiliar que definimos anteriormente para procesar el mensaje del usuario llamando a `openai_process_message(user_message)` y almacenando la respuesta en `openai_response_text`. A continuación, limpiaremos esta respuesta para eliminar cualquier línea vacía utilizando una simple función de una línea en python, `os.linesep.join([s for s in openai_response_text.splitlines() if s])`.

Una vez que tengamos esta respuesta limpia, utilizaremos otra función auxiliar que definimos anteriormente para convertirla en voz. Por lo tanto, llamaremos a `text_to_speech` y pasaremos los dos parámetros requeridos que son `openai_response_text` y `voice`. Almacenaremos el valor de retorno de la función en una variable llamada `openai_response_speech`.

Dado que `openai_response_speech` es un tipo de datos de audio, no podemos enviarlo directamente dentro de un json ya que este solo puede almacenar datos textuales. Por lo tanto, utilizaremos algo llamado *"codificación base64"*. En palabras simples, podemos convertir cualquier tipo de datos binarios a una representación textual codificando los datos en formato base64. Así que simplemente utilizaremos `base64.b64encode(openai_response_speech).decode('utf-8')` y almacenaremos el resultado nuevamente en `openai_response_speech`.

Ahora tenemos todo listo para nuestra respuesta, así que finalmente utilizaremos la misma función `app.response_class` y enviaremos los tres parámetros requeridos. El `status` y el `mimetype` serán exactamente los mismos que definimos en nuestra anterior `speech_to_text_route`. En la respuesta, utilizaremos la función `json.dumps` como lo hicimos antes y pasaremos un diccionario como parámetro que contenga `"openaiResponseText":openai_response_text` y `"openaiResponseSpeech":openai_response_speech`.

Luego retornamos la `response`.

***NOTA :* Puede que encuentres errores de indentación en archivos de Python. Asegúrate de que tu código utilice un espaciado consistente (sin mezclar tabs y espacios) y verifica la indentación cuidadosamente antes de ejecutar la aplicación.**

The content of this lab is licensed under Apache 2.0

Paso 8: Probar tu asistente personal

El asistente ya está completo y listo para usar.

Ahora que hemos actualizado el código de manera considerable, es un buen momento para reconstruir nuestra imagen de Docker y probar que esté funcionando como se espera en este entorno.

Suponiendo que las URLs del modelo de Texto a Voz y Voz a Texto estén configuradas correctamente, solo necesitas reconstruir la imagen de la aplicación y volver a ejecutarla para que tenga todos los últimos cambios.

Este paso asume que no tienes ningún contenedor en ejecución para la aplicación. Si lo tienes, presiona `Ctrl (^)` y `C` al mismo tiempo para detener el contenedor.

bash

```
docker build . -t voice-chatapp-powered-by-openai
docker run -p 8000:8000 voice-chatapp-powered-by-openai
```

Run

Luego, simplemente abre la aplicación en una nueva pestaña (o si ya tienes la pestaña abierta, actualiza esa página).

Open App

Recuerda probar todas las diferentes entradas y características. Por ejemplo, escribir un mensaje usando el cuadro de mensaje y usar el micrófono.

Nota: El navegador puede bloquear la pestaña de la aplicación para abrirse o solicitar acceso al micrófono. Por favor, permite las ventanas emergentes y concede permisos de micrófono para que la aplicación funcione correctamente.

The content of this lab is licensed under Apache 2.0

Crea un Asistente de Voz con el Modelo GPT de OpenAI y IBM Watson



Tiempo estimado necesario: 1 hora

Introducción

¡Bienvenido a este proyecto guiado sobre la creación de un asistente de voz utilizando las bibliotecas de voz de OpenAI e IBM Watson para incrustar! El proyecto guiado te llevará a construir un asistente virtual que puede recibir entrada de voz, convertirla a texto utilizando tecnología de reconocimiento de voz, enviar el texto al modelo GPT de OpenAI, recibir una respuesta, convertirla a voz utilizando tecnología de texto a voz y, finalmente, reproducirla al usuario. El asistente de voz tendrá un front-end receptivo utilizando HTML, CSS y JavaScript, y un back-end confiable utilizando Flask.

Haz clic aquí para jugar con una demostración de la aplicación final que crearás.

Al final del curso, tendrás un profundo entendimiento de los asistentes de voz y las habilidades para crear tu propio asistente potenciado por IA que puede comunicarse a través de entrada y salida de voz. También tendrás una sólida base en desarrollo web utilizando Python, Flask, HTML, CSS y JavaScript, ¡y una impresionante aplicación de pila completa terminada!

Antes de comenzar, proporcionemos un poco de contexto sobre cada tema.

OpenAI

OpenAI es una organización de investigación que tiene como objetivo promover y desarrollar inteligencia artificial amigable de una manera que beneficie a la

The content of this lab is licensed under Apache 2.0

son modelos de procesamiento de lenguaje natural de vanguardia. Utilizarás GPT en tu asistente para permitirle entender y responder a una amplia gama de entradas de usuario.

Bibliotecas de voz de IBM Watson para incrustar

Las bibliotecas de voz de IBM Watson® para incrustar son un conjunto de bibliotecas de texto a voz y de voz a texto en contenedores diseñadas para ofrecer a nuestros socios de IBM mayor flexibilidad para incorporar lo mejor de la tecnología de IBM Research® en sus soluciones. Ahora disponibles como IA incrustable, los socios obtienen mayores capacidades para construir aplicaciones de transcripción de voz y síntesis de voz más rápidamente y desplegarlas en cualquier entorno híbrido de múltiples nubes. Estas tecnologías permiten que el asistente se comunique con los usuarios a través de entrada y salida de voz.

Asistentes de voz

Un asistente virtual es un programa diseñado para simular la conversación con usuarios humanos, especialmente a través de Internet utilizando la voz humana natural. Los asistentes pueden ser utilizados en una variedad de industrias, incluyendo servicio al cliente, comercio electrónico y educación.

Python (Flask)

Python es un lenguaje de programación popular que se utiliza ampliamente en el desarrollo web y la ciencia de datos. Flask es un marco web para Python que facilita la construcción de aplicaciones web. Utilizarás Python y Flask para construir el backend de tu asistente de voz. Python es un lenguaje poderoso que es fácil de aprender y tiene un gran ecosistema de bibliotecas y marcos que se pueden aprovechar en proyectos como el tuyo.

HTML - CSS - Javascript

HTML (Lenguaje de Marcado de Hipertexto) es un lenguaje de marcado utilizado para estructurar contenido en la web. CSS (Hojas de Estilo en Cascada) es un lenguaje de hojas de estilo utilizado para describir la apariencia y el formato de un documento escrito en HTML. Javascript es un lenguaje de programación que se utiliza comúnmente para agregar interactividad a las páginas web. Juntas, estas tecnologías nos permiten construir un frontend visualmente atractivo e interactivo para tu asistente. Los usuarios podrán interactuar con el asistente de voz a través

The content of this lab is licensed under Apache 2.0

Objetivos de aprendizaje

Al final de este proyecto, podrás:

- Explicar los conceptos básicos de los asistentes de voz y sus diversas aplicaciones
- Configurar un entorno de desarrollo para construir un asistente utilizando Python, Flask, HTML, CSS y Javascript
- Implementar la funcionalidad de voz a texto para permitir que el asistente entienda la entrada de voz de los usuarios
- Integrar el asistente con el modelo GPT de OpenAI para darle un alto nivel de inteligencia y la capacidad de entender y responder a las solicitudes de los usuarios
- Implementar la funcionalidad de texto a voz para permitir que el asistente se comunique con los usuarios a través de salida de voz
- Combinar todos los componentes anteriores para crear un asistente funcional que pueda recibir entrada de voz y proporcionar una respuesta hablada
- (Opcional) Desplegar el asistente en un servidor web para su uso por un público más amplio

Requisitos previos

Tener conocimientos de los conceptos básicos de HTML/CSS, Javascript y Python es deseable, pero no esencial. Haremos nuestro mejor esfuerzo para explicar cada paso del proceso, así como cualquier código mostrado a lo largo del camino.

Paso 1: Entendiendo la in… →

The content of this lab is licensed under Apache 2.0